

Web Security - SOP

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

User Agents

- They are the client side component responsible for the retrieval and display of web resources
 - wget, curl, etc.
 - Chrome, Firefox, etc.
- Some User Agents support the execution of client side code
 - Java Applets
 - ActiveX Controls
 - JavaScript

JavaScript

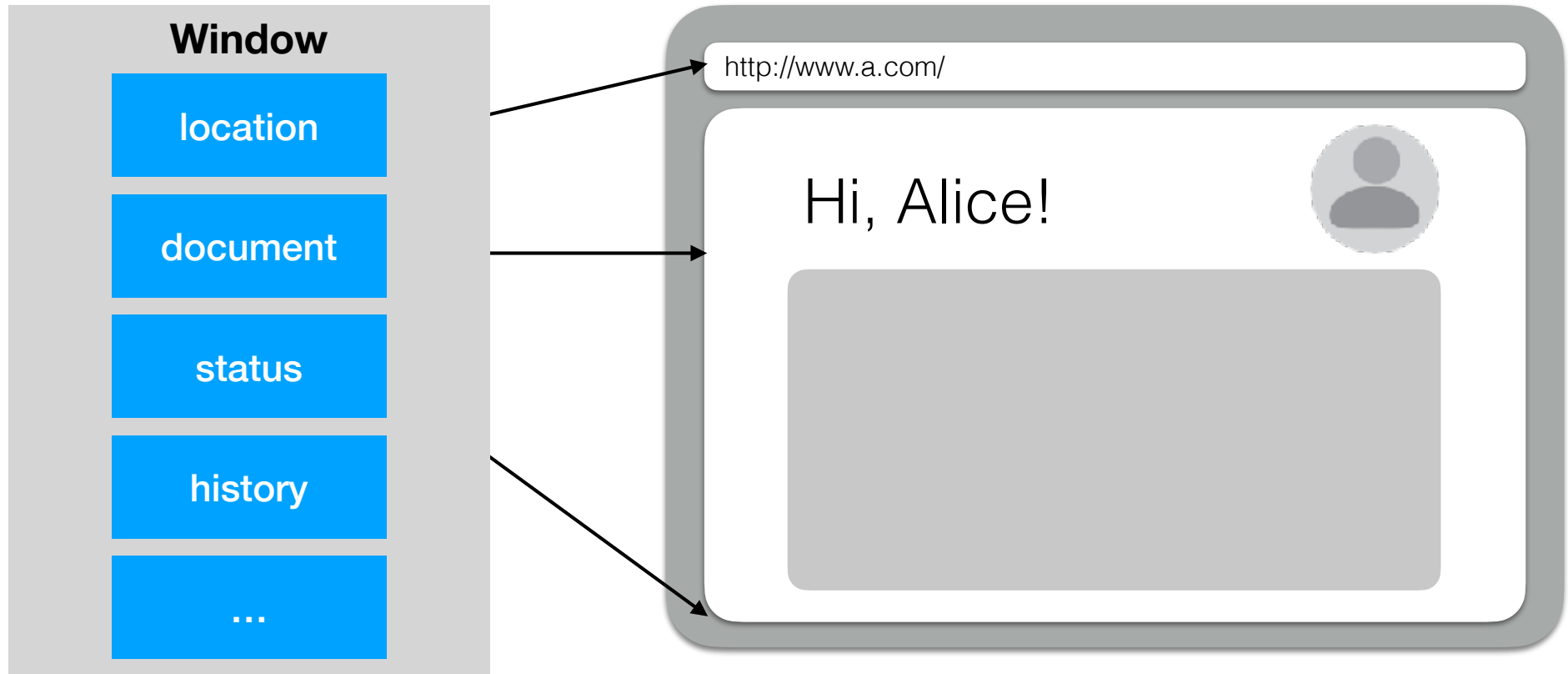
- A scripting language used to implement interactive and dynamic behavior in web pages
- Known as the client-side scripting language
- It was initially developed at Netscape in 1995, called LiveScript
- JScript is Microsoft's version
- EcmaScript is a standardized version of JavaScript
- VBScript is based on Microsoft Visual Basic

它是整个浏览器对象模型的根对象，其他对象如document、location等都是它的子对象，通过它可以访问和控制浏览器的大部分功能。

Client-Side JavaScript

window是浏览器的默认全局对象，所有在全局作用域中定义的变量、函数等都属于window对象的属性和方法。

- The APIs that allow you to control (nearly) everything in a web client, *i.e.*, a web browser.
- Client-side JavaScript models a browser in an Object-Oriented way



window is the default global object

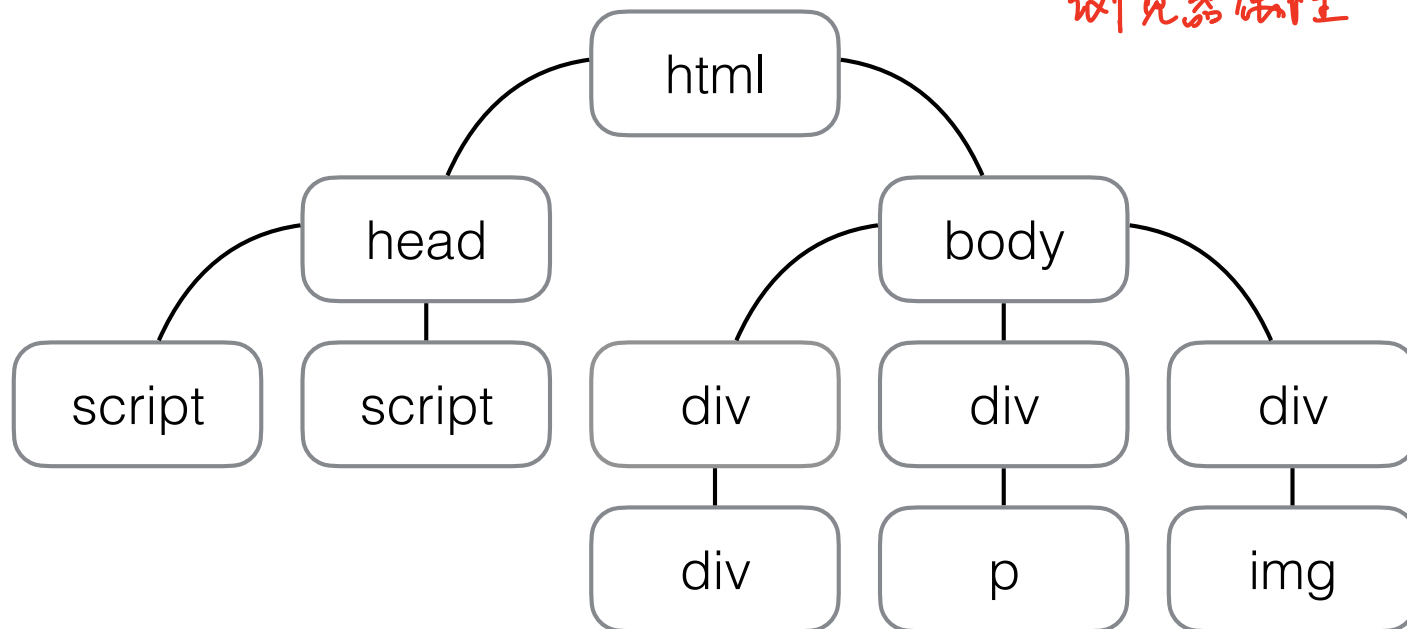
Document Object Model

DOM是html和xml文档的编程接口，这个API将html文档视为树结构

- A cross-platform and language independent API that treats an HTML document as a tree structure.
- Each node in the DOM tree is an object representing a part of the document.
- Browser Object Model (BOM) is an API to the browser properties

这些对象:

浏览器属性

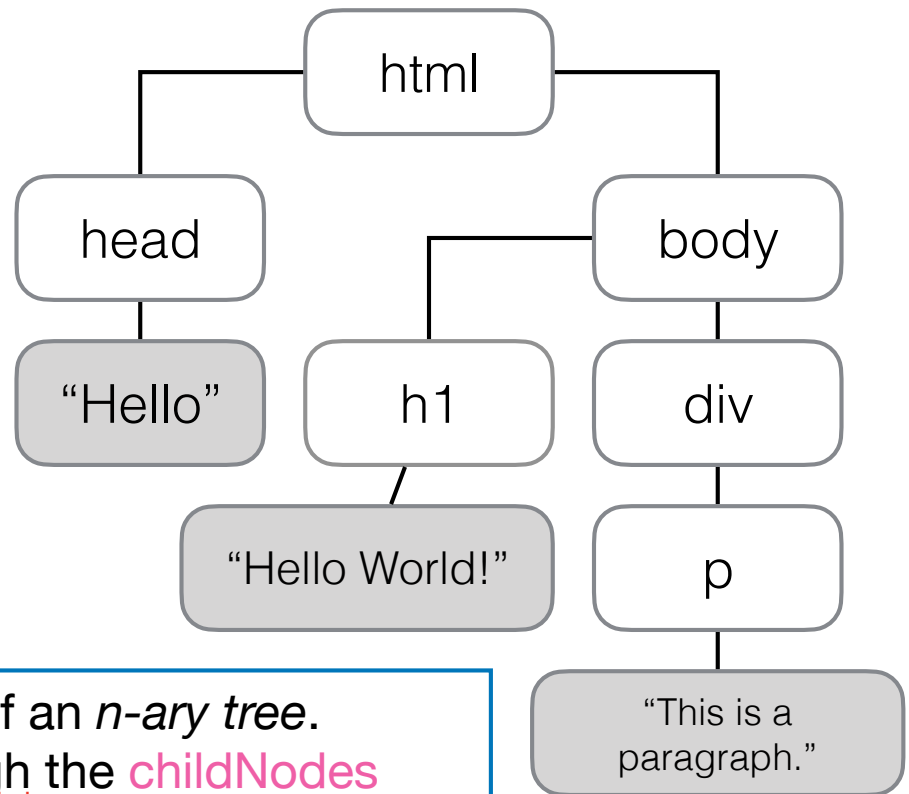


They are objects, and instances of **Node**.

DOM Tree

Each HTML element is represented by a DOM object.

```
<html>
  <head>
    <title>Hello</title>
  </head>
  <body>
    <h1>Hello World!</h1>
    <div>
      <p>This is a paragraph.</p>
    </div>
  </body>
</html>
```



每个节点可以有任意数量的子节点

- Every node in the DOM tree is the root of an *n*-ary tree.
- Each node's children is accessed through the **childNodes** property.
DOM 树中每个节点子节点都可以通过该节点的 **childNodes** 属性来访问
- Some nodes are leaf nodes, i.e., they do not have child.
 - text, comment

JavaScript Security

1. 无需预先编译 - 直接交给引擎执行
2. 动态解析 - 边解析语法边执行
3. 灵活但需谨慎 - 注意变量/函数提升等特性

- JavaScript code is downloaded either as part of or with the HTML code by the browser
- It is executed ^{即时编译} on-the-fly and in the context of the embedding web page JavaScript代码在它所嵌入的网页环境中执行
- JavaScript is very powerful
 - Can access the DOM in arbitrary ways
 - Read content, insert new content, delete existing content
 - Read and write to Cookies, which are critical data for user session management

On - the - fly

JavaScript Security (Cont.)

- The security of JavaScript code execution is guaranteed by a sandboxing mechanism
 - No access to files and network resources
 - No window smaller than 100x100 pixels
 - No access to the browser's history
- The details of sandbox implementation is browser specific

同源策略

Same-Origin Policy

“源”由三个部分组成，就像一个网络身份证：

复制

协议://域名:端口

- 协议：http、https、ftp等
- 域名：包括主域名和子域名
- 端口：默认http是80，https是443

示例判断

URL1	URL2	是否同源	原因
https://example.com/a.html	https://example.com/b.html	✓	完全相同
http://example.com	https://example.com	✗	协议不同
https://a.example.com	https://b.example.com	✗	子域名不同
https://example.com:8080	https://example.com	✗	端口不同

这个是一个网站的不同页面

框架隔离

Frame Isolation

☆ 这里访问的 web page 可能同源, 也有可能不同源

- Goals:
 - It is safe to visit an arbitrary (unsafe) web page
 - It is safe to visit two web pages at the same time
 - It is safe to visit a page embedding other frames
- Need:
 - Each web site is isolated from all other websites
 - Web pages of the same web site are not isolated

安全访问嵌入其它框架的页面

当网页A内嵌了网页B的iframe时，浏览器会确保：

- 1. 父页面A不会被内嵌的B破坏
- 2. 内嵌的B页面不会被父页面A恶意操控
- 3. 不同源的iframe之间互相隔离

实际案例

html复制

```
<!-- 新闻网站嵌入Youtube视频 -->
<iframe src="https://youtube.com/embed/123"></iframe>
```

▶ 运行 HTML

- ☒ 正常情况：Youtube播放器正常工作，但无法读取新闻网站的数据
- ☐ 如果没有隔离：恶意视频网站可以通过iframe窃取你的新闻网站登录状态

意思解读

同一个网站（相同协议+域名+端口）的不同页面：

- 1. 可以相互访问DOM（如通过window.open()）
- 2. 共享Cookie/LocalStorage等存储
- 3. 部分浏览器特性可共用（如Service Worker）

特性	同源页面	跨源iframe
隔离级别	不隔离（视为同一应用）	完全隔离
DOM访问	自由访问	完全禁止（除非CORS授权）
典型场景	SPA多标签页、后台管理系统	嵌入第三方地图/支付/广告

完全隔离 ≠ 交互不可控

1. 广告iframe的隔离机制

html复制

```
<!-- 抖音页面内嵌拼多多广告 -->
<iframe
  src="https://ads.pinduoduo.com/123"
  sandbox="allow-scripts allow-popups"
></iframe>
```

▶ 运行 HTML

隔离维度	实现方式	安全效果
DOM隔离	抖音无法读取/修改拼多多iframe内部的DOM	防止抖音窃取拼多多的广告点击数据
Cookie隔离	拼多多广告只能访问 pinduoduo.com 的Cookie，无法读取 douyin.com 的Cookie	防止拼多多获取抖音的登录状态
进程隔离	Chromium浏览器中，抖音主页面和拼多多iframe运行在不同进程	即使拼多多广告崩溃，也不会影响抖音主页面

- 1. 隔离是绝对的：拼多多广告iframe与抖音主页面完全隔离，杜绝数据泄露
- 2. 交互是可控的：通过浏览器安全策略允许用户主动触发的跳转
- 3. 用户主权保留：最终是否跳转由用户决定（可见地址栏变化/弹窗提示）

Frame Isolation (Cont.)

- Frame-Frame relationships
 - `canScript`(A, B)
 - Can Frame A execute a script that access any DOM elements of Frame B?
 - `canNavigate`(A, B)
 - Can Frame A change the origin of content of Frame B?
- Frame-Principal relationships
 - `readCookie`(F, D), `writeCookie`(F, D)
 - Can Frame F read/write cookies of domain D?

维度	Frame (框架/iframe)	Domain (域名)
定义	HTML文档中的嵌入式窗口 (实际存在的DOM元素)	网站的地址标识 (逻辑概念)
表现形式	<code><iframe></code> 标签	<code>example.com</code> 、 <code>sub.example.com</code>
核心作用	在页面中嵌套显示其他内容	标识网站身份, 用于安全策略判断
创建方式	通过HTML代码或JavaScript动态创建	通过DNS注册获得

虽然A和B都是HTML代码, 但浏览器会根据**代码被哪个域加载**来赋予不同的“身份凭证”。

类比解释:

- 两个Python脚本 (`.py` 文件) :
 - 文件A由用户Alice拥有
 - 文件B由用户Bob拥有
 - **即使代码内容相同**, 系统会根据文件所有者决定权限

mermaid

复制

flowchart TD

A[HTML文件] --> B{被哪个域加载}

B -->|https://a.com加载| C[打上a.com的“数字身份证”]

B -->|https://b.com加载| D[打上b.com的“数字身份证”]

Frame-Frame relationships (框架间关系)

canScript(A, B)

1. canScript(A, B)

判断框架A是否能通过脚本访问框架B的DOM

mermaid

复制

flowchart LR; A["A[Frame A]"] -->|JavaScript尝试访问| B["B[Frame B的DOM]"]; B --> C["C{同源检查}"]; C -->|同源| D["D[允许访问]"]; C -->|不同源| E["E[抛出SecurityError]"];

不同源就不可以访问!

canNavigate(A, B)

2. canNavigate(A, B)

判断框架A是否能导航(改变)框架B的内容

mermaid

复制

flowchart TD; A["A[Frame A]"] -->|尝试修改URL| B["B[Frame B]"]; B --> C["C{是否同源或受控白名单}"]; C -->|是| D["D[允许有限导航]"]; C -->|否| E["E[禁止导航]"];

当网站A要嵌入网站B的iframe时，可以通过 `<iframe sandbox>` 属性主动声明允许哪些跨源操作：

html

复制

<!-- 网站A嵌入网站B时，选择性开放权限 --> <iframe src="https://siteB.com" sandbox="allow-scripts allow-popups allow-forms" ></iframe>

▶ 运行 HTML

沙箱白名单权限表

权限值	允许的操作	安全风险
allow-scripts	运行JavaScript	中
allow-popups	打开新窗口（如拼多多广告跳转）	低
allow-forms	提交表单	中
allow-same-origin	突破同源限制（慎用！）	高
allow-top-navigation	允许iframe导航父页面（极危险！）	极高

Frame-Principal relationships

框架与域的关系

1. Frame F 与 Domain D 同源

定义：
F 的源（`protocol + hostname + port`）与 Cookie 的 Domain 属性完全匹配或符合子域规则。

规则：

操作	是否允许	条件
读取Cookie	✔ 允许	Cookie 未设置 <code>HttpOnly</code> 标志
写入Cookie	✔ 允许	符合以下任一条件： - Cookie 的 <code>Domain</code> 包含 F 的域（如 <code>.example.com</code> ） - 未设置 <code>Domain</code> （默认为当前域）

2. Frame F 与 Domain D 不同源

定义：
F 的源与 Cookie 的 Domain 不匹配（跨域或跨子域且未显式共享）。

规则：

操作	是否允许	条件
读取Cookie	✗ 禁止	无论 Cookie 如何设置，均无法通过 <code>document.cookie</code> 读取
写入Cookie	✗ 禁止	除非同时满足： 1. Cookie 设置 <code>SameSite=None; Secure</code> 2. 通过 CORS 显式授权

Same-Origin Policy

- The Same-Origin Policy
 - An essential security mechanism that isolates different pages
 - A frame in one origin shall not be allowed to access resources of frames in a different origin
 - JavaScript code can access only resources that are associated with the same origin
 - Cannot make HTTP requests to, access the DOM, Cookies and local storage of a site/frame of a different origin

1. 禁止跨源框架访问资源

例子：

- 你在 `微博.com` 的页面中内嵌了一个 `淘宝.com` 的广告iframe。
- **限制：** 微博的JavaScript代码无法直接读取或修改这个淘宝iframe里的内容（如商品价格、用户登录状态等）。

2. JavaScript只能访问同源资源

例子：

- 你在 `知乎.com` 的网页里写了一段JavaScript，想读取 `B站.com` 的用户评论数据。
- **限制：** 浏览器会直接阻止这段代码获取B站的数据，因为两者域名不同。

3. 禁止跨源HTTP请求

例子：

- 你的网站 `小A.com` 尝试用JavaScript向 `银行.com` 发起转账请求。
- **限制：** 浏览器会拦截这个请求，防止恶意网站偷偷操作你的银行账户。

4. 禁止跨源访问DOM

例子：

- `新闻网站.com` 的页面里嵌入了 `视频网站.com` 的播放器iframe。
- **限制：** 新闻网站无法用JavaScript直接控制视频播放器的播放/暂停按钮。

5. 禁止跨源读取Cookie/本地存储

例子：

- 你登录了 `邮箱.com`，然后在另一个标签页打开 `黑客网站.com`。
- **限制：** 黑客网站的JavaScript无法读取你邮箱的登录Cookie，防止会话劫持。

Same-Origin Policy (Cont.)

- Origin: It is the granularity of protection of the SOP
 - A tuple of **protocol**, **hostname** and **port**
 - Determined by string matching
 - Every frame has an origin
 - An <iframe>'s origin is determined by the URL where it is loaded, not the URL of its parent frame
 - JavaScript code executes under the origin of the embedding frame!
 - This opens the door for many other security problems on the Web

特性	说明	示例
字符串精确匹配	区分大小写，必须完全一致	<code>https://Example.com</code> $\neq$ <code>https://example.com</code> (主机名大小写不同)
iframe源独立于父页面	iframe的源由其 <u>src 属性</u> 决定，与父页面无关	父页面 <code>a.com</code> 内嵌 <code><iframe src="b.com"></code> → iframe的源是 <code>b.com</code>
JS代码执行身份绑定	脚本的权限取决于它被 <u>哪个域的页面加载</u> ，而非代码本身内容	相同的JS文件在 <code>a.com</code> 和 <code>b.com</code> 下运行时，拥有不同的权限

1. iframe的源判定

html

复制

```
<!-- 父页面: https://social.com -->
<iframe src="https://ads.com"></iframe>
```

▶ 运行 HTML

- iframe的源 = `https://ads.com`
- 与父页面 `social.com` 不同源 → 禁止互相访问DOM/Cookie

2. JavaScript执行身份

- 代码来源决定权限:

假设有一段JS代码:

javascript

复制

```
function stealData() {
  console.log(document.cookie);
}
```

- 在 `bank.com` 运行时: 能读取该银行的Cookie
- 在 `hacker.com` 运行时: 只能读取黑客网站的Cookie

PPT最后一句指出的问题:

"This opens the door for many other security problems on the Web"

原因:

- 当合法网站 (如 `trusted.com`) 被注入恶意JS代码时, 这些代码会继承 `trusted.com` 的高权限
- 例如:

html

复制

```
<!-- 被攻击的trusted.com页面 -->
<script>
  // 恶意代码! 但因加载自trusted.com, 能读取该域下的敏感数据
  fetch('/user/data').then(data => sendToHacker(data));
</script>
```

▶ 运行 HTML

SOP: XMLHttpRequests

- XMLHttpRequests (XHR) allows JavaScript to send HTTP requests to a server, and receive data in the response
- SOP prohibits XHRs from being sent to a different origin
- JavaScript can read only responses from the same origin

JavaScript 只能读取来自与当前网页相同来源（同源）的 HTTP 响应数据

SOP不禁止 发送 跨源XHR请求，但禁止JavaScript 读取 跨源响应的内容
(除非服务器通过CORS显式允许)

SOP: HTTP Responses

- A frame can load Images, CSS, Fonts from another origin
 - It cannot inspect the content of the resources
 - Similar to loading an iFrame from another origin
- A frame can load JavaScript from another origin
 - It cannot inspect the content of the script
 - It can interact with the script
 - For example, calling a function defined in a script loaded from another origin

1. 静态资源（图片/CSS/字体）

- 允许加载：

html

复制

```
<!-- 跨域图片 -->


<!-- 跨域CSS -->
<link rel="stylesheet" href="https://cdn.com/style.css">
```

▶ 运行 HTML

- 禁止检查内容：

javascript

复制

```
// 尝试读取跨域图片像素数据
const img = document.querySelector('img');
const canvas = document.createElement('canvas');
canvas.getContext('2d').drawImage(img, 0, 0);
// 报错: Tainted canvases may not be exported
```

类比：

你可以把别家的书（图片/CSS）摆在自己书架上，但不能拆开书页查看内容（像素数据/CSS规则文本）。

2. JavaScript资源

- 允许加载+执行：

html

复制

```
<script src="https://cdn.com/library.js"></script>
```

▶ 运行 HTML

- 库中定义的函数可直接调用：

javascript

复制

```
// 假设library.js定义了函数
importedFunction(); // 允许调用
```

- 禁止读取源码：

javascript

复制

```
fetch('https://cdn.com/library.js')
  .then(res => res.text()) // 被SOP阻止！
  .catch(err => console.error(err));
```

类比：

你可以用别家工厂的工具（JS函数），但无法查看他们的设计图纸（源码）。

Relaxing the SOP

因为主机名不同
(www. 前缀)
被视为不同源

- Problem: `https://www.facebook.com` and `https://facebook.com` are two different origins
- 通过设置相同的父域名，让子域名间建立信任关系：
Solution: The two frames can set their domain to the same value, i.e., `facebook.com`
 - `a.domain.com` -> `domain.com`
 - `b.domain.com` -> `domain.com`
 - `a.domain.com` -> `com`
 - Cannot set domain to a public suffix
 - `user.github.com` -> `github.com`
 - Both frames need to set domain to the same value

一、允许的操作 (子域名→父域名)

1. 同级子域互通

javascript

复制

```
// 在 https://a.domain.com 和 https://b.domain.com 中均设置:  
document.domain = "domain.com";
```

- 效果: **a.domain.com** 和 **b.domain.com** 将被视为同源
- 用途: 电商主站与支付子站共享登录状态

2. GitHub Pages 案例

javascript

复制

```
// 在 https://user.github.io 中设置:  
document.domain = "github.io";
```

- 效果: 允许同用户的多个Pages项目通信 (如 **user.github.io/project1** 和 **user.github.io/project2**)

二、禁止的操作

1. 尝试设置公共后缀

javascript

复制

```
// 在 https://a.domain.com 中:  
document.domain = ".com"; // 报错! 禁止设置为顶级域
```

- 原因: 否则 **bank.com** 和 **evil.com** 可伪装成同源 **.com**, 造成安全灾难

2. 单方面设置无效

javascript

复制

```
// 仅在 https://a.domain.com 中设置:  
document.domain = "domain.com";  
// 但 https://b.domain.com 未设置 → 仍然跨域
```

Relaxing the SOP (Cont.)

- Additional solutions:
 - Cross-Origin Resource Sharing: *CORS 跨域资源共享*
 - *Access-Control-Allow-Origin*: <http://www.foo.com>
 - *Access-Control-Allow-Origin*: *
 - Cross-document messaging:
 - Sender: `targetWindow.postMessage(Message, targetOrigin, [transfer])`
 - Receiver: `window.addEventListener("message", receiveMessage, false);`
 - *receiveMessage* takes an *event* parameter that carries the origin of the sender

一、CORS（跨域资源共享）

1. 核心机制

服务器通过HTTP响应头声明允许哪些外部源访问资源：

http

复制

```
Access-Control-Allow-Origin: http://www.foo.com # 允许特定域
# 或
Access-Control-Allow-Origin: * # 允许所有域（慎用！）
```

2. 工作流程

mermaid

复制

```
sequenceDiagram
    Browser->>Server: 跨域请求（Origin: http://your-site.com）
    Server-->>Browser: Access-Control-Allow-Origin: http://your-site.com
    Browser->>Your JS: 允许读取响应
```

3. 关键场景

- 前端：`https://shop.com` 请求 `https://api.shop.com`
- 后端配置：

http

复制

```
Access-Control-Allow-Origin: https://shop.com
Access-Control-Allow-Methods: GET, POST
```

二、跨文档消息通信（postMessage）

1. 核心API

javascript

复制

```
// 发送方（iframe父页面）
iframe.contentWindow.postMessage("秘密数据", "https://trusted.com");

// 接收方（iframe内部）
window.addEventListener("message", (event) => {
  if (event.origin !== "https://parent.com") return;
  console.log("收到消息:", event.data);
});
```

2. 安全要点

参数	作用	示例
<code>targetOrigin</code>	指定目标窗口的源（精确匹配）	<code>"https://bank.com"</code>
<code>event.origin</code>	验证消息来源（防止伪造）	<code>if (event.origin !== expectedOrigin) return;</code>

Credits

- Some slides are based on the slides of Giovanni Vigna